

03 · Einführung in Echtzeitbetriebssysteme

FreeRTOS und Task-Management · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 51

Willkommen in der Profiliga!

- Herzlich willkommen zum 3. Termin unserer Vorlesung
- Letzte Woche: Die Mathematik (RMS, EDF) und unser eigener kleiner Scheduler
- Heute: Wir werfen unseren Eigenbau weg und wechseln in die Industrie-Praxis
- Wir führen FreeRTOS ein — das am weitesten verbreitete RTOS für Mikrocontroller
- Ziel: Verstehen, wie das OS unter der Haube Speicher und CPU verwaltet

Notizen

Guten Morgen und herzlich willkommen zurück! Letzte Woche: Scheduling-Schranken und kooperativer Round-Robin-Scheduler. Heute werfen wir den Eigenbau weg — didaktisch wertvoll, für die Industrie nicht ausreichend. FreeRTOS: wie Tasks verwaltet werden, was Hardware-Ressourcen kostet, und wie wir es auf dem STM32H755 einsetzen.

Folie

2 / 51

Agenda für heute

1. Recap: Warum unser Eigenbau-Scheduler letzte Woche gescheitert ist
2. Vom Bare-Metal zum RTOS (Latenz vs. Determinismus)
3. Die Architektur von FreeRTOS
4. Task-Management: TCB und Speicherbedarf
5. Die State-Machine (Ready, Running, Blocked, Suspended)
6. Tick-Rate, Delays und Context Switch
7. CMSIS-RTOS v2 API und Labor am STM32H755

Notizen

Kurz Labor-Rückblick, dann Trade-off Latenz vs. Determinismus. Danach FreeRTOS: Task-Modell, RAM-Bedarf, Zustandsmaschine, Timing und CMSIS — bevor wir im Labor flashen.

Recap: Unser Round-Robin Eigenbau

- Letzte Woche: Array aus Funktionszeigern (TCBs)
- Scheduler lief ringförmig durch alle Tasks (Round-Robin)
- Kontextwechsel als einfacher C-Aufruf: `task_list[idx].task_func()`;
- Voraussetzung: System war kooperativ

```
// Die Task MUSS das OS rufen!  
void Task_1() {  
    while(1) {  
        LED_On();  
        os_yield(); // Freiwillig!  
    }  
}
```

Notizen

Kleines OS in C: Array speichert Task-Funktionen. `os_yield()` erhöht den Index und ruft die nächste Funktion. Elegant ohne Assembler — aber kooperativ: jede Task musste brav `os_yield()` aufrufen.

Recap: Der Crash im Labor

- In der Labor-Aufgabe haben wir die Kooperation sabotiert
- Task 2: gigantische, blockierende Zählschleife ohne `os_yield()`
- Ergebnis:
 - Task 1 (LED) ist eingefroren
 - Der Scheduler wurde nie wieder aufgerufen
 - Das System war tot (Starvation)
- Fazit: Für harte Echtzeit dürfen wir der Anwendungssoftware niemals vertrauen — wir brauchen Präemption

Notizen

Mutwillige Sabotage: Zählschleife ohne `os_yield()`. LED stand still, MCU schien eingefroren — Main-Loop in Task 2 gefangen. Harte Echtzeit braucht Präemption durch Hardware-Interrupts.

Die Notwendigkeit eines echten RTOS

- Ein modernes System (z. B. Dual-Core STM32H755) hat komplexe Anforderungen:
 - Harte Timing-Garantien (Präemption)
 - Sichere Kommunikation zwischen Tasks (Queues)
 - Absicherung gemeinsamer Ressourcen (Mutexes)
 - Speichermanagement
- Das baut man nicht an einem Nachmittag
- Wir greifen auf bewährte, zertifizierbare Software zurück

Notizen

Bastel-Scheduler reicht nicht: präemptives Scheduling (SysTick/PendSV), sichere IPC, Mutexes. In der Industrie nutzt man etablierte Open-Source- oder kommerzielle RTOS.

Was leistet ein RTOS?

- Multitasking (Scheduling): Orchestrierung nach strengen Regeln (z. B. RMS)
- Inter-Process Communication (IPC): Queues und Mailboxes
- Synchronisation: Mutexes und Semaphore für Interrupt-Sicherheit
- Memory Management: RAM-Stacks für jede Task
- Hardware Abstraction Layer (HAL): ARM-Spezifika (SysTick, PendSV) — generischer C-Code darüber

Notizen

Fünf Kernaufgaben: Scheduling, IPC, Synchronisation, Speichermanagement, Abstraktion. FreeRTOS-Code auf Cortex-M4 lässt sich oft auf ESP32 oder RISC-V portieren.

Der Markt für Echtzeitbetriebssysteme

Kommerziell (oft Safety-zertifiziert):

- VxWorks: Luft- und Raumfahrt (Mars Rover)
- QNX: Auto-Infotainment
- AUTOSAR OS: Steuergeräte (meist OSEK-basiert)

Open Source / industrie-nah:

- Zephyr: Linux Foundation (z. B. Nordic)
- FreeRTOS: Marktführer bei kleineren Mikrocontrollern

Notizen

Kommerziell: VxWorks (NASA), QNX, AUTOSAR OS. Open Source: Zephyr stark im Kommen. Unangefochtener Platzhirsch für Mikrocontroller — und unser System: FreeRTOS.

Lernziele des heutigen Tages

- Trade-off Latenz vs. Determinismus verstehen
- Task-Modell von FreeRTOS implementieren können
- Verstehen, warum eine Task im Zustand Blocked exakt 0% CPU benötigt
- RAM-Bedarf eines Task-Sets und Tick-Auflösung berechnen
- CMSIS-RTOS v2 statt nativer FreeRTOS-API nutzen
- FreeRTOS in der STM32CubeIDE konfigurieren und Präemption im Labor prüfen

Notizen

RTOS macht den MCU zunächst „langsam“, schenkt aber Vorhersagbarkeit. Architektur, Zustände, Timing/CMSIS und Prioritäten — dann Labor auf dem Cortex-M4.

Architektur: Bare-Metal vs. RTOS

Bare-Metal (Super-Loop)

- Hardware → Ihr C-Code
- Sehr flache Architektur
- Ein einziger Call-Stack im RAM
- CPU rechnet oft dauerhaft (Polling, $\approx 100\%$ Auslastung)

RTOS-Architektur

- Hardware → RTOS → Ihre Tasks
- OS als Zwischenschicht
- Jede Task hat einen eigenen Call-Stack
- CPU kann schlafen (Idle-Task), wenn nichts zu tun ist

Notizen

Bare-Metal: C direkt auf Hardware, ein Stack, oft 100% Polling-Last. RTOS: Schicht dazwischen, eigener Stack pro Task, Schlafmodus wenn alle warten.

Das Kernparadigma: Latenz vs. Determinismus

- Vorurteil: „Wir können kein RTOS nutzen, das ist zu langsam!“
- Fakt: Ein RTOS kostet Rechenzeit (Overhead)
 - SysTick-Interrupt typisch alle 1 ms
 - Scheduler muss Listen sortieren (Taktzyklen)
 - Context Switch (PendSV) kostet ca. 100–300 Takte
- Aber: Wir tauschen maximale Geschwindigkeit gegen garantierte Pünktlichkeit (Determinismus)

Notizen

Technisch richtig, konzeptionell gefährlich: RTOS kostet Takte (SysTick, Scheduler, PendSV). Absolute Latenz wird etwas länger — dafür Determinismus: ausrechenbare Pünktlichkeit statt wackeliger Höchstgeschwindigkeit.

Visualisierung: Jitter-Vergleich

- Hardware-Event triggert eine Reaktion



- Bare-Metal: oft schneller, aber unregelmäßig (hoher Jitter)
- RTOS: etwas später (OS-Overhead), bei hoher Priorität nahezu konstant (Jitter ≈ 0)

Notizen

Oben: externes Ereignis zweimal. Bare-Metal ohne Interrupts: schnell wenn die Superloop gerade pollt, sonst ewig — gewaltiger Jitter. RTOS: minimale Startlatenz, aber konstant durch Präemption. In der Regelungstechnik Gold wert.

Einführung in FreeRTOS

- Ursprünglich von Richard Barry (2003) entwickelt
- Open Source (MIT-Lizenz — auch in kommerziellen, geschlossenen Produkten nutzbar)
- 2017 von Amazon (AWS) übernommen
- Eigenschaften:
 - Winziger Footprint (Scheduler ab ≈ 4 KB ROM, < 1 KB RAM)
 - Sehr portabel (8-Bit AVR bis 32-Bit ARM Cortex-A)
 - Quasi-Standard in Maker- und Industrie-Welt (ESP32, STM32)

Notizen

MIT-Lizenz: ändern und kommerziell nutzen ohne Veröffentlichungspflicht. Amazon übernahm 2017 für AWS-IoT. Kernstärke: Footprint und Portabilität — beherrschen Sie FreeRTOS, beherrschen Sie einen Großteil des Embedded-OS-Marktes.

Die FreeRTOS Architektur

- Das gesamte Betriebssystem besteht aus nur wenigen C-Dateien
- Kern-Dateien (OS-Logik, chip-unabhängig):
 - `tasks.c`: Scheduler, TCB-Verwaltung, Ready-Listen
 - `list.c`: Optimierte doppelt verkettete Listen
 - `queue.c`: Inter-Task-Kommunikation
- Port-Layer (hardware-spezifisch):
 - `port.c/portmacro.h`: Assembler für SysTick und PendSV (Cortex-M4)

Notizen

Kein Blackbox-Kernel wie Linux: Kern in Standard-C (`tasks.c`, `list.c`, `queue.c`). Hardware-Brücke im Port-Layer (`ARM_CM4F/port.c`) mit PendSV-Assembler.

Zusammenfassung: Warum ein RTOS?

Vorteile:

- Vorhersagbares Verhalten (Determinismus) durch hardwarenahe Präemption
- Saubere Software-Struktur (unabhängige Tasks statt Spaghetticode)
- Fertige, getestete Module für Kommunikation und Speicherschutz

Nachteile:

- Verlust absoluter CPU-Rohleistung (Scheduler-Overhead)
- Höherer RAM-Verbrauch (eigener Stack pro Task)
- Komplexere Fehlersuche (Race Conditions)

Notizen

Determinismus und Struktur gegen Overhead, RAM und Race Conditions. Als Nächstes: wie FreeRTOS Tasks und RAM verwaltet. Fragen zum Architektur-Teil?

Das FreeRTOS Task-Modell

- Eine Task in FreeRTOS ist eine C-Funktion
- Wichtigste Regel: Die Funktion darf niemals ans Ende gelangen (`return` ist verboten)
- Als unendliche Schleife `for(;;)` oder `while(1)` schreiben

```
void vMyFirstTask(void *pvParameters) {
    // 1. Initialisierung (läuft nur einmal)
    Init_Sensor();

    // 2. Die Endlosschleife (der Task-Körper)
    for(;;) {
        Read_Sensor();
        vTaskDelay(pdMS_TO_TICKS(100)); // OS: schlafe 100 ms
    }
    // HIER DARF DER CODE NIEMALS ANKOMMEN!
}
```

Notizen

Gewöhnliche C-Funktion mit `void *`-Parameter. Niemals `return` oder Ende der Funktion — sonst Hard Fault. Setup einmal, dann ewige `for(;;)` mit Arbeit und Schlafen.

Der Task Control Block (TCB) in FreeRTOS

- FreeRTOS legt für jede Task ein Struct im RAM an
- Wichtige Felder in `tskTCB`:
 - `pxTopOfStack`: Wo liegen die CPU-Register dieser Task?
 - `xStateListItem`: Knoten für Ready/Blocked-Listen
 - `uxPriority`: Aktuelle Priorität
 - `pcTaskName`: String für Debugging (z. B. "MotorTask")
- Der TCB wird vom OS verwaltet — wir fassen ihn niemals direkt an

Notizen

Bei `xTaskCreate` alloziert FreeRTOS den TCB: Stack-Pointer für PendSV, Listen-Knoten, Name für Debugger. Nur über die FreeRTOS-API manipulieren.

Speichermanagement: Der Stack

- In Bare-Metal: ein globaler Stack für das ganze Programm
- In FreeRTOS: jede Task hat einen eigenen, isolierten Stack im RAM
- Lokale Variablen und Funktionsaufrufe nutzen nur diesen privaten Stack
- Vorsicht: Stack zu klein → Stack Overflow in den RAM anderer Tasks

Notizen

Zehn Tasks = zehn separate RAM-Blöcke. Lokale Variable landet nur im eigenen Block. Zu große lokale Arrays: Stack Overflow — Absturzursache Nummer 1 in der Praxis.

Berechnung des RAM-Bedarfs

- RAM ist auf Mikrocontrollern kostbar (STM32H755: 1 MB; kleinere Cortex-M oft nur 16 KB)
- RAM-Kosten einer neuen Task:

$$\text{RAM} = \text{TCB} (\approx 80 \text{ Byte}) + (\text{Stack-Wörter} \times 4 \text{ Byte})$$

- Beispiel: Stack von 128 Wörtern auf 32-Bit-ARM = 512 Byte
 - Gesamt: $\approx 512 + 80 = 592$ Byte pro Task
- Lektion: Keine großen Arrays als lokale Variablen — Heap oder globale Variablen nutzen

Notizen

FreeRTOS-API: Stackgröße 128 meint 128 Wörter à 4 Byte = 512 Byte. Zehn Tasks: fast 6 KB ohne sinnvolle Arbeit. Große Puffer nie lokal in der Task.

Speicherallokation: Statisch vs. Dynamisch

Dynamisch (`xTaskCreate`):

- OS ruft intern `pvPortMalloc()` (RTOS-Heap)
- Problem: Kann fehlschlagen, wenn der Heap voll ist (Laufzeitfehler)

Statisch (`xTaskCreateStatic`):

- Globale C-Arrays für den Stack übergeben
- Vorteil: RAM-Mangel fällt schon beim Kompilieren auf (Automotive-Standard)

Notizen

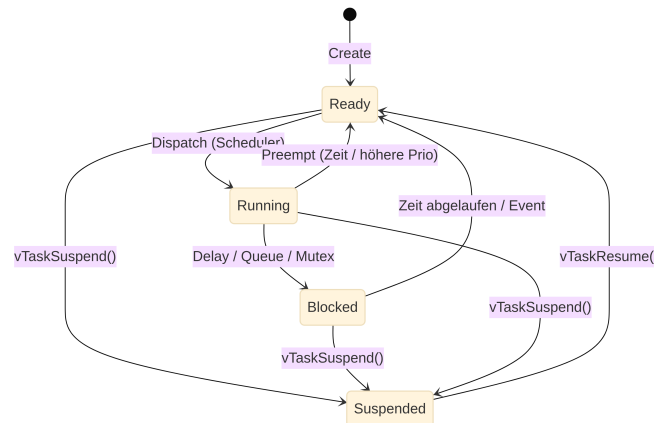
Dynamisch bequem, aber gefährlich bei vollem Heap. Safety/Automotive oft nur statisch: Linker bricht ab, bevor das Auto startet. Im Labor zunächst der dynamische Weg.

Folie

20 / 51

Das FreeRTOS Zustandsmodell

- Das Verständnis dieses Modells entscheidet, ob Ihr System echtzeitfähig ist
- Eine Task befindet sich immer in exakt einem von vier Zuständen



Notizen

Herzstück: State-Diagramm. Blocked ist kriegsentscheidend. Nie Blocked → Running direkt — immer erst Ready. Suspended: künstliches Koma bis `vTaskResume()`.

Folie

21 / 51

Der „Running“-Zustand

- Die Task besitzt die Kontrolle über die CPU-Hardware (Register, ALU)
- Auf einem Single-Core kann immer nur genau eine Task Running sein
- Auf dem STM32H755 (Dual-Core):
 - Ein Kernel auf Cortex-M7 (eine Task Running)
 - Ein Kernel auf Cortex-M4 (eine Task Running)
- Die Task bleibt Running, bis sie:
 - etwas Blockierendes aufruft (Delay, Mutex)
 - von einer wichtigeren Task unterbrochen wird (Präemption)

Notizen

Running = König der CPU. Heute Nachmittag FreeRTOS nur auf dem M4. Illusion der Gleichzeitigkeit

durch schnelles Umschalten. Thronverlust: freiwillig (Delay) oder gewaltsam (SysTick/höhere Prio).

Folie

22 / 51

Der „Ready“-Zustand

- Warteschlange der arbeitswilligen Tasks
- Logisch und rechnerisch bereit, alle nötigen Daten vorhanden
- Warum läuft sie nicht?
 - Eine Task mit gleicher oder höherer Priorität blockiert aktuell Running
- FreeRTOS verwaltet pro Prioritäts-Level eine eigene Ready-Liste

Notizen

Ready = kein Problem, nur gerade nicht ausgewählt. Array von Listen pro Priorität: Scheduler schaut nur auf die oberste nicht-leere Liste — immer gleich viele Takte (Determinismus).

Folie

23 / 51

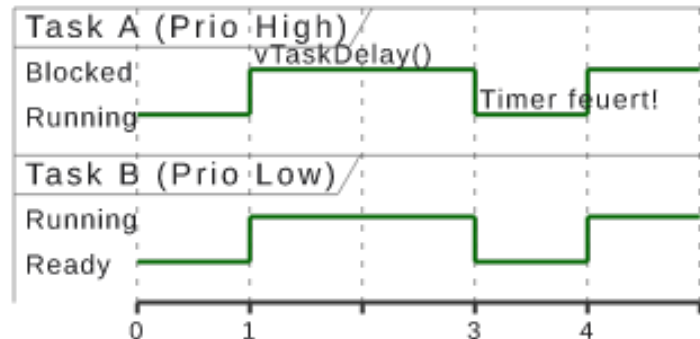
Der „Blocked“-Zustand (Die Magie des RTOS)

- Wichtigster Zustand für ein stabiles Echtzeitsystem
- Task wartet auf ein Ereignis:
 - Zeitlich: `vTaskDelay(100)` (schlafe exakt 100 Ticks)
 - Logisch: Taste, UART-Daten, Queue
- Clou: Blocked-Task wird vom Scheduler beim Suchen komplett ignoriert
- Verbrauch: 0,00% CPU-Rechenzeit
- Deshalb niemals `HAL_Delay()` in einem RTOS (verbrennt CPU in Zählschleife)

Notizen

Warten auf ADC? FreeRTOS-Blockade: CPU weg, Blocked-Liste, null Prozent Last. `HAL_Delay` lässt die CPU kreisen und bleibt Running — immer RTOS-Blockade-Funktionen nutzen.

Visualisierung: Task-Zustände im Zeitverlauf



- Task A schickt sich selbst in den Blocked-Zustand
- Task B darf endlich rechnen
- Nach 2 ms weckt der OS-Timer Task A auf — sie verdrängt B sofort (Präemption)

Notizen

t=0: A rechnet, B wartet. t=1: A ruft `vTaskDelay` — B bekommt die CPU. t=3: Schlafzeit vorbei, A zurück in Ready mit höherer Prio — Präemption von B.

FreeRTOS Scheduler-Konfiguration

- Scheduler-Verhalten tiefgreifend anpassbar
- Steuerung über `FreeRTOSConfig.h`
- Präemption aktivieren:
 - `configUSE_PREEMPTION = 1`: Scheduler darf Tasks unterbrechen (RMS-Stil)
 - `configUSE_PREEMPTION = 0`: kooperatives Scheduling (wie unser Eigenbau)
- Für unsere Vorlesung: diesen Wert immer auf 1 setzen

Notizen

`FreeRTOSConfig.h` ist die Schaltzentrale. `configUSE_PREEMPTION = 0` macht FreeRTOS kooperativ wie unser Eigenbau. Für ernsthafte Projekte und den Rest der Vorlesung strikt auf 1.

Prioritäten in FreeRTOS (Achtung: Falle!)

- Jede Task bekommt beim Start eine Priorität als Integer (`uint8_t`)
- FreeRTOS-Regel:
 - Prio 0 = niedrigste Priorität
 - Prio `configMAX_PRIORITIES - 1` = höchste Priorität
- Falle für Hardware-Entwickler:
 - ARM Cortex-M NVIC denkt genau andersherum (0 = höchste Prio)
- Merken: OS-Level (hohe Zahl = wichtig), Hardware-Level (kleine Zahl = wichtig)

Notizen

FreeRTOS: 0 unwichtig, hohe Zahl wichtig. NVIC: 0 ist König. Strikt trennen: OS-C-Code vs. Hardware-Register — bringt Anfänger oft zur Verzweiflung.

Die Idle-Task (Prio 0)

- Was passiert, wenn alle eigenen Tasks Blocked sind?
- Die CPU muss Assembler ausführen — sie kann nicht „nichts“ tun
- FreeRTOS erstellt beim Start automatisch die Idle-Task
- Immer Priorität 0 (lässt jeden anderen ran)
- Nutzen der Idle-Task:
 - Speicher gelöschter Tasks aufräumen
 - CPU in Hardware-Tiefschlaf (`WFI` — Wait For Interrupt), starker Stromspar-Effekt (`configUSE_TICKLESS_IDLE`)

Notizen

Drei Tasks Blocked: CPU braucht trotzdem Befehle. Idle-Task mit Prio 0 läuft nur, wenn niemand sonst will. Modern: `WFI` schaltet den Kern ab bis zum nächsten Timer — Batteriesensoren über Jahre.

Strom sparen: Tickless Idle

- Was passiert, wenn alle Tasks für z. B. 500 ms blockiert sind?
- Normalerweise wacht der SysTick trotzdem jede 1 ms auf, prüft Listen und schläft wieder
- Problem: Die CPU wird 500-mal sinnlos geweckt (Stromverschwendung)
- Lösung: `configUSE_TICKLESS_IDLE = 1`
 - FreeRTOS berechnet die Zeit bis zum nächsten Event (z. B. 500 ms)
 - SysTick aus, Low-Power-Timer auf diese Dauer programmieren
 - CPU geht in echten Deep-Sleep

Notizen

Im Normalfall feuert der SysTick jede Millisekunde, auch wenn alle Tasks eine halbe Sekunde blockiert sind — 500 sinnlose Aufwachzyklen. Mit `configUSE_TICKLESS_IDLE` berechnet der Scheduler die Zeit bis zur nächsten Deadline, schaltet den 1-ms-SysTick ab, programmiert einen stromsparenden Hardware-Timer und schickt die CPU schlafen. Für batteriebetriebene Sensoren entscheidend.

Der RTOS-Herzschlag: Die Tick-Rate

- Das Zeitgefühl von FreeRTOS basiert auf Ticks
- Ein Tick entspricht einem Interrupt des SysTick-Timers
- Gesteuert durch `configTICK_RATE_HZ` (meist 1000 Hz)
- Bedeutung:
 - Bei 1000 Hz dauert 1 Tick exakt 1 ms
 - Das OS löst Zeiten nur in ganzzahligen Vielfachen von Ticks auf
 - Ein Delay von 1,5 ms ist nicht möglich (wird gerundet)

Notizen

`configTICK_RATE_HZ` steht fast immer auf 1000: SysTick jede Millisekunde. FreeRTOS hat keine Nanosekunden-Uhr — die kleinste Einheit ist der Tick. Für 1,5 ms Delay geht das nicht; Mikrosekunden-präzises Timing braucht einen separaten Hardware-Timer, kein OS-Delay.

Zeit-Umrechnung (Die goldene Regel)

- Zeiten niemals als nackte Integer angeben
- `vTaskDelay(100)` bedeutet: schlafe 100 Ticks
- Ändert ein Kollege die Tick-Rate auf 100 Hz?
 - Aus 100 ms wird plötzlich 1 s

Immer das Makro nutzen:

```
// FALSCH (Tick-abhängig):  
vTaskDelay(100);  
  
// RICHTIG (immer 100 ms):  
vTaskDelay(pdMS_TO_TICKS(100));
```

Notizen

Die FreeRTOS-API erwartet Ticks. Bei 1000 Hz sind 100 Ticks zufällig 100 ms. Senkt jemand die Rate auf 100 Hz, blockiert dieselbe Zahl eine ganze Sekunde. Deshalb immer `pdMS_TO_TICKS()` — der Precompiler rechnet Millisekunden in Ticks um.

Der Context Switch (Overhead-Analyse)

- Der Wechsel erfolgt via PendSV-Assembler-Code
- Was kostet das die CPU?
 - Hardware rettet 8 Kern-Register (≈ 12 Takte)
 - PendSV-Handler rettet restliche Register (Push)
 - Scheduler-Logik (Listen umhängen)
 - Neue Register laden (Pop)
- Summe: ca. 150–300 Taktzyklen (je nach Optimierung)
- Bei 100 MHz CPU-Takt: nur $\approx 2\text{--}3$ s

Notizen

PendSV sichert Register, FreeRTOS-Assembler pusht den Rest, Listen werden umgehängt, neue Register geladen. Auf Cortex-M4 grob 150–300 Takte; bei ≥ 100 MHz am STM32H755 also 2–3 Mikrosekunden. Der Latenz-Overhead ist real, für die meisten Anwendungen vernachlässigbar.

Wann wechselt der Scheduler?

- Der Context Switch passiert nicht nur beim 1-ms-SysTick
- Das OS wechselt sofort asynchron, wenn:
 - eine laufende Task `vTaskDelay()` aufruft (freiwillig abgeben)
 - ein Hardware-Interrupt (z. B. UART RX) Daten in eine Queue legt und dadurch eine blockierte Task mit höherer Prio aufweckt
- Genau das sorgt für minimale Latenzen in harten Echtzeitsystemen

Notizen

Fehlannahme: Scheduler wechselt nur im 1-ms-Raster. Für harte Echtzeit zu langsam. `vTaskDelay` löst sofort `PendSV` aus. UART-ISR legt ein Byte in eine Queue: hochpriorisierte wartende Task wird Ready; nach der ISR springt die CPU nicht zurück in die alte Task, sondern in die aufgeweckte.

`vTaskDelay` vs. `HAL_Delay`

- Das wichtigste Konzept für das Labor

Metrik	<code>HAL_Delay(100)</code>	<code>vTaskDelay(...)</code>
CPU-Zustand	Leere <code>while</code> -Schleife	Wechsel zu anderer Task / Idle
Task-Zustand	Bleibt Running	Wechselt nach Blocked
Auslastung U	100% CPU-Last	0% CPU-Last
Fazit	Zerstört Echtzeit	Echtes OS-Scheduling

Notizen

`HAL_Delay` ist Bare-Metal: wartet auf SysTick, hält die CPU aber in einer leeren Schleife. Das OS denkt, die Task rechnet. `vTaskDelay` sagt: fertig für 100 ms — Task blockiert, Auslastung nahe null, andere Tasks dürfen laufen. Im Labor: niemals `HAL_Delay` in FreeRTOS-Tasks.

Drift-freies Timing: vTaskDelayUntil

- vTaskDelay blockiert relativ zur aktuellen Zeit
- Problem: Ausführungszeit verschiebt den nächsten Start (Drift)
- Lösung: vTaskDelayUntil() merkt sich den absoluten Startzeitpunkt
- Wichtig für präzise PID-Regler

```
TickType_t lastWakeTime;  
lastWakeTime = xTaskGetTickCount();  
  
for(;;) {  
    Rechne_PID(); // Dauer schwankt!  
  
    // Startet exakt alle 100 ms,  
    // ohne zeitlichen Drift:  
    vTaskDelayUntil(  
        &lastWakeTime,  
        pdMS_TO_TICKS(100)  
    );  
}
```

Notizen

vTaskDelay verzögert relativ zum Aufruf: 2 ms rechnen + 10 ms schlafen = Periode 12 ms; schwankt die Rechenzeit, driftet der Zyklus. PID braucht konstantes Δt . vTaskDelayUntil nimmt den letzten Aufwach-Zeitpunkt und schläft nur die Restzeit — kein Drift über Stunden.

Die Datei FreeRTOSConfig.h

- Jeder Port von FreeRTOS benötigt diese Konfigurationsdatei
- Definiert das Verhalten des OS zur Kompilierzeit
- Wichtige Makros:
 - configUSE_PREEMPTION (1 = an, 0 = aus)
 - configMAX_PRIORITIES (Anzahl der Listen, oft 7)
 - configMINIMAL_STACK_SIZE (Mindest-RAM für die Idle-Task)
 - configTOTAL_HEAP_SIZE (RAM-Pool für dynamische Allokation)

Notizen

Zuerst in `FreeRTOSConfig.h` schauen: präemptiv?, Prioritätsstufen?, Heap-Größe? `configTOTAL_HEAP_SIZE` entscheidet, ob Task-Erstellung zur Laufzeit fehlschlägt. In der STM32CubeIDE oft über das grafische Interface statt Handedit.

Folie

36 / 51

Zusammenfassung: Scheduler und Timing

- FreeRTOS ist primär präemptiv (SysTick, PendSV)
- Die Tick-Rate bestimmt die feinste zeitliche Auflösung (meist 1 ms)
- Blocked-Tasks entlasten die CPU massiv
- Konfiguration statisch über C-Makros
- Prioritäten: hohe Zahl = hohe Prio (Achtung Cortex-M NVIC)

Notizen

Zusammenfassung Theorie: portables, präemptives OS; SysTick für Zeit, PendSV für Registertausch; blockierte Task kostet null Rechenzeit; Prioritäts-Fallstricke merken. Als Nächstes: Wie ruft man FreeRTOS im C-Code auf?

Folie

37 / 51

FreeRTOS API vs. CMSIS-RTOS v2

- Native FreeRTOS-API ist historisch gewachsen (z. B. `xTaskCreate`)
- Problem: Vendor Lock-in — Wechsel zu Zephyr oder ThreadX wird teuer
- Lösung: CMSIS-RTOS v2 (ARM)
 - Standardisierte Abstraktionsschicht (Wrapper)
 - Einheitliche C-Funktionen (z. B. `osThreadNew`)
 - Darunter kann ein beliebiges RTOS liegen

Notizen

FreeRTOS-Namen mit `x/v/pC` (ungarische Notation). Reine FreeRTOS-API kettet an den Vendor. CMSIS-RTOS ist ein Wörterbuch: `osThreadNew` wird intern zu `xTaskCreate`. OS wechseln = Kern tauschen, Applikationscode bleibt.

STMicroelectronics und CMSIS

- Die STM32CubeIDE generiert standardmäßig CMSIS-RTOS v2
- FreeRTOS dient uns als Backend

Aktion	FreeRTOS nativ	CMSIS-RTOS v2
Task erstellen	<code>xTaskCreate()</code>	<code>osThreadNew()</code>
Task blockieren	<code>vTaskDelay()</code>	<code>osDelay()</code>
Queue anlegen	<code>xQueueCreate()</code>	<code>osMessageQueueNew()</code>
Zeit holen	<code>xTaskGetTickCount()</code>	<code>osKernelGetTickCount()</code>

Notizen

CubeIDE stülpt den CMSIS-v2-Wrapper über FreeRTOS. Vorlesung und Labor: strikt CMSIS. Statt `vTaskDelay` schreiben wir `osDelay` — kürzer, portabler, gleiche Wirkung.

Ein CMSIS-Thread im Code

- Thread-Attribute: Name, Stack-Größe, Prio
- Danach Anmeldung beim OS
- Falle: CMSIS erwartet Stack-Größe in Bytes, nicht in Wörtern

```
const osThreadAttr_t myTaskAttr = {
    .name = "MyTask",
    .stack_size = 1024, // Bytes!
    .priority = osPriorityNormal,
};

osThreadId_t myTaskHandle =
    osThreadNew(
        MyTask_Function,
        NULL,
        &myTaskAttr
    );
```

Notizen

CMSIS nennt Tasks Threads. Struct mit Name, Stack und Prio. Falle: 1024 bedeutet 1 KB Bytes; nativ FreeRTOS hätte 256 Wörter verlangt. `osThreadNew` legt TCB an und setzt Ready.

Starten des Betriebssystems

- Nach Hardware-Init und Task-Anlage: Kontrolle abgeben

```
osKernelStart();
```

- Was passiert?
 - SysTick wird auf 1 ms programmiert
 - Scheduler durchsucht die Ready-Liste
 - Stack-Pointer springt in die Task mit höchster Prio
- Wichtigste Regel: Code nach `osKernelStart()` wird niemals ausgeführt

Notizen

Point of no Return: `osKernelStart()` (nativ: `vTaskStartScheduler()`). SysTick, Sprung in den ersten TCB, kein Return. Ein `printf` darunter erscheint nie — der lineare Bare-Metal-Pfad ist beendet.

Fehlerbehandlung: Wenn RAM fehlt

- Erschöpfter Heap: `osThreadNew` liefert NULL
- Immer prüfen, sonst Hard Fault zur Laufzeit

```
if (myTaskHandle == NULL) {  
    // Kein RAM mehr für TCB oder Stack  
    Error_Handler();  
}
```

- Labor-Tipp: Task startet nicht? `configTOTAL_HEAP_SIZE` in CubeMX erhöhen

Notizen

Zu viele Tasks: dynamische Allokation schlägt fehl, Handle ist NULL. Professionell abfangen (rote LED, Stop). Im Debugger springt eine Task nicht an? Meist voller Heap — Gesamtgröße erhöhen.

Schritt 1: STM32CubeIDE-Setup

- Ziel: FreeRTOS auf dem Cortex-M4, präemptives RMS live beweisen
- Neues Projekt STM32H755 (Cortex-M4 aktivieren)
- Middleware: FREERTOS, Interface CMSIS_V2
- `TOTAL_HEAP_SIZE` mindestens 16384 Bytes (16 KB)

Notizen

Nicht mehr selbst basteln: echtes RTOS per CubeIDE. Middleware FreeRTOS, CMSIS_V2, Heap sofort auf 16 KB — sonst knallt es beim Anlegen mehrerer Tasks.

Schritt 2: Der Konflikt der Timer

- Problem:
 - HAL nutzt SysTick für `HAL_Delay()`
 - FreeRTOS braucht SysTick zwingend für den Scheduler
 - Beide auf demselben Timer: Absturz zur Laufzeit
- Lösung (CubeMX-Warnung!):
 - System Core → SYS → Timebase Source
 - Von SysTick auf TIM6 umstellen
 - HAL nutzt TIM6; SysTick gehört FreeRTOS exklusiv

Notizen

CubeIDE warnt sofort: HAL und FreeRTOS kämpfen um SysTick. Lösung: HAL-Timebase auf TIM6. SysTick frei für den Scheduler.

Schritt 3: Task-Design für das Labor

- Drei Tasks, um RMS-Scheduling zu beweisen:
 - Heartbeat (`osPriorityNormal`): LED alle 500 ms via `osDelay`
 - HeavyMath (`osPriorityBelowNormal`): Endlosschleife ohne Delay, 100% CPU
 - Emergency (`osPriorityHigh`): wartet auf Tastendruck (später per Queue)

Notizen

Heartbeat: normale Prio, blinkt alle 500 ms. HeavyMath: niedrigere Prio, reine while-Schleife ohne Delay. Emergency: höchste Prio, meist blockiert auf Taste. Drei Tasks im Configurator anlegen und Code generieren.

Code-Struktur nach Generierung

- CubeIDE platziert CMSIS-Aufrufe (`osThreadNew`) in `main.c`
- Unten: leere Skelette für die Task-Funktionen

```
/* USER CODE BEGIN Header_StartHeartbeat */
void StartHeartbeat(void *argument)
{
    /* USER CODE BEGIN StartHeartbeat */
    for(;;)
    {
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
        osDelay(500); // Ticks; bei 1000 Hz = 500 ms
    }
    /* USER CODE END StartHeartbeat */
}
```

Notizen

In `main.c` ganz unten: generierte Task-Skelette. Heartbeat: Toggle + `osDelay(500)`. Bei `configTICK_RATE_HZ = 1000` sind das 500 ms. CMSIS arbeitet in Ticks; die goldene Regel `pdMS_TO_TICKS` gilt für die native FreeRTOS-API. Diese Task verbringt fast die gesamte Lebenszeit im Blocked-Zustand.

Code-Skelett: Die HeavyMath-Task

- Diese Task bricht die Blockade-Regel absichtlich

```
void StartHeavyMath(void *argument)
{
    volatile uint32_t primes_found = 0;
    for(;;)
    {
        // KEIN osDelay! 100% CPU.
        // In Bare-Metal: System freezt.
        Calculate_Primes(&primes_found);
    }
}
```

Notizen

Primzahlen, Hashes oder Counter — Variable `volatile`, sonst optimiert der Compiler die Schleife weg. Kein `osDelay`: Running wird nie freiwillig verlassen. Beim kooperativen Scheduler von letzter Woche wäre hier alles eingefroren.

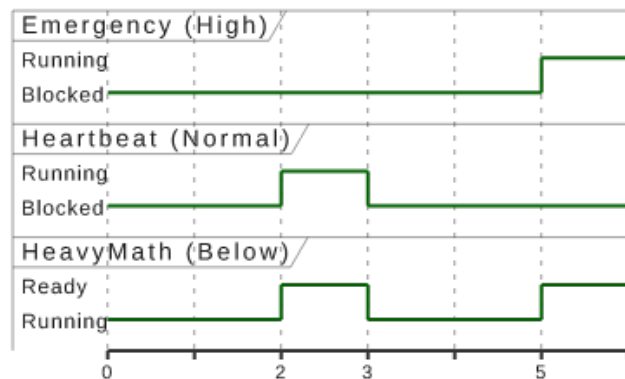
Beobachtung 1: Der Präemptions-Beweis

- Code auf den STM32H755 flashen
- Was passiert?
 - Heartbeat-LED blinkt stabil im 500 ms-Takt
- Warum friert das System nicht ein?
 - Heartbeat: Normal; HeavyMath: BelowNormal
 - Nach 500 ms weckt SysTick Heartbeat
 - Höhere Prio: OS reißt HeavyMath die CPU weg (Präemption)

Notizen

LED blinkt trotz 100%-Mathe-Schleife. Heartbeat schläft Blocked; HeavyMath nutzt Idle-Zeit. SysTick weckt Heartbeat; PendSV wirft Mathe von der CPU, Toggle, wieder Delay, zurück zur Mathematik.

Visuelle Erklärung: Präemption im Labor



- CPU dauerhaft bei 100%, Auslastung aber nach Prioritäten (RMS) aufgeteilt

Notizen

t=0: Emergency und Heartbeat Blocked, HeavyMath Running. t=2: Heartbeat-Timer, Präemption. t=3: Blinken fertig, wieder Delay, HeavyMath weiter. t=5: Emergency-Button, Prio High übernimmt sofort. RMS auf Hardware-Ebene.

Beobachtung 2: Starvation durch falsche Prioritäten

- Absichtlicher Fehler zum Verständnis von RMS:
 - HeavyMath-Prio $\rightarrow$ `osPriorityHigh` (höher als Heartbeat)
- Flashen und beobachten:
 - LED bleibt dunkel / blinkt nie wieder
 - Heartbeat verhungert in der Ready-Liste (Starvation)
 - Hochpriore Endlosschleife gibt die CPU nicht frei

Notizen

HeavyMath auf High: LED tot. Heartbeat wacht auf, Scheduler sagt: Prio Normal gegen High — warten. Mathe-Schleife endet nie $\rightarrow$ Starvation. Das ist kein Priority Inversion (dort hält niedrige Prio eine Sperre); hier reine Verhungern durch falsche Prioritäten. Scharfes Schwert!

Debugging: Stack Overflow erkennen

- Board tut nichts (keine LED, kein sichtbarer Code)?
- Oft: Stack der Task zu klein (Stack Overflow)
- Hardware-Tipp:
 - Cortex-M4 löst HardFault aus
 - Debugger pausieren: Ausführung steckt in `HardFault_Handler()`
- Lösung: In CubeMX Stack Size der betroffenen Task verdoppeln

Notizen

90 % der „nichts tut“-Fälle: Stack Overflow durch lokale Variablen. Hardware wirft HardFault; Call-Stack zeigt `HardFault_Handler`. Cube-Configurator: Stack verdoppeln.

Zusammenfassung des Tages

- RTOS: maximale Geschwindigkeit (Latenz) gegen Garantie (Determinismus)
- Jede Task braucht RAM-Budget (TCB + eigener Stack)
- Blockierte Task: 0% Rechenzeit — niemals reine C-Delays
- CMSIS-RTOS v2 schützt vor Vendor Lock-in
- Timer-Interrupts und Prioritäten verhindern das Einfrieren wichtiger Teile

Notizen

Sie verstehen den Latenz-Determinismus-Trade-off, das RAM-Modell und CMSIS als API. Im Labor: präemptives Scheduling live und die Gefahr falsch gesetzter Prioritäten. Vielen Dank — bis gleich im Labor!